

CSA1717-ARTIFICIAL INTELLIGENCE
CO2-AT3-DRY RUN TEST
SOLUTION

Name: Roshitha

Reg No: 192424400

QUESTION 1 – DETAILED SOLUTION

Solution: A* Search Algorithm

A* Search is an informed search technique that finds the shortest path between a start node and a goal node by combining the actual distance already travelled with an estimate of the distance still remaining. Unlike uninformed methods such as Breadth-First Search or Depth-First Search, A* uses a heuristic to steer the search toward the goal, which usually means far fewer nodes need to be examined before the optimal path is found.

Every node is scored using the evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual cost accumulated from the start node to n , $h(n)$ is the heuristic estimate of the remaining cost from n to the goal, and $f(n)$ is their sum – the estimated total cost of a path that passes through n .

The problem graph has six nodes: A, B, C, D, E and G. The heuristic values supplied for this problem are:

Node $h(n)$

A 7

B 6

C 4

D 3

E 2

G 0

The task is to find the shortest path from the Start Node (A) to the Goal Node (G).

Initial State

Open List : A

Closed List : Empty

$g(A) = 0$, $h(A) = 7$, so $f(A) = 0 + 7 = 7$. Since A is the only node in the Open List, it is chosen for expansion.

Iteration 1

Current Node : A

A has two neighbours – B, reached at edge cost 2, and C, reached at edge cost 4.

For B: $g(B) = 0 + 2 = 2$, $h(B) = 6$, $f(B) = 2 + 6 = 8$.

For C: $g(C) = 0 + 4 = 4$, $h(C) = 4$, $f(C) = 4 + 4 = 8$.

Both B and C are added to the Open List:

Open List : B(8), C(8)

Closed List : A

B and C are tied on $f(n)$; B is expanded next since it was generated first.

Iteration 2

Current Node : B

B's neighbours are C, D, and E.

Node C: a new path gives $g(C) = 2 + 3 = 5$, which is worse than the existing $g(C) = 4$, so C's recorded cost is left unchanged.

Node D: $g(D) = 2 + 7 = 9$, $h(D) = 3$, $f(D) = 9 + 3 = 12$.

Node E: $g(E) = 2 + 2 = 4$, $h(E) = 2$, $f(E) = 4 + 2 = 6$.

Open List : E(6), C(8), D(12)

Closed List : A, B

E now has the smallest $f(n)$, so E is expanded next.

Iteration 3

Current Node : E

E's neighbours are B, C, and D. B is already in the Closed List and is skipped.

Node C: new cost $g(C) = 4 + 3 = 7$, which is worse than the recorded $g(C) = 4$, so no update is made.

Node D: new cost $g(D) = 4 + 2 = 6$, which improves on the previous $g(D) = 9$. D's cost is updated: $h(D) = 3$, $f(D) = 6 + 3 = 9$.

Open List : C(8), D(9)

Closed List : A, B, E

C now has the minimum $f(n)$ and is expanded next.

Iteration 4

Current Node : C

C's neighbours are A, B, and E – all of which are already closed or already have a better recorded cost, so no changes are made to the Open List.

Open List : D(9)

Closed List : A, B, C, E

Iteration 5

Current Node : D

D's neighbours are B, E, and G. B and E are already closed. G has not been reached before, so:

$g(G) = 6 + 2 = 8$, $h(G) = 0$, $f(G) = 8 + 0 = 8$.

Open List : G(8)

Closed List : A, B, C, D, E

Iteration 6

Current Node : G

G is the goal node, so the search terminates here.

Optimal Path

Tracing back through the recorded parent of each node gives the final shortest path:

A → B → E → D → G

Path Cost Calculation

A → B = 2

B → E = 2

E → D = 2

D → G = 2

Total Cost = 2 + 2 + 2 + 2 = 8

Explanation of Open List and Closed List

At every iteration, the Open List holds every node that has been discovered but not yet expanded, while the Closed List holds every node that has already been fully expanded. A* always removes the node with the smallest $f(n)$ from

the Open List, which is what makes the search consistently explore the most promising direction first rather than exploring nodes purely by discovery order or by depth.

Analysis

The heuristic values decrease steadily as the search gets closer to G, which is what allows A* to prefer E and then D over repeatedly revisiting A, B and C. Because the heuristic never overestimates the true remaining distance (it is admissible), A* is guaranteed to return the optimal path – and it does so here by expanding only 6 nodes rather than exhaustively exploring every possible route through the graph.

Final Result

Optimal Path : A → B → E → D → G

Total Path Cost : 8

The A* Search Algorithm identifies this shortest path efficiently, revising D's recorded cost mid-search once a cheaper route through E is discovered, and terminating as soon as the goal is popped with the smallest $f(n)$ on the Open List.

QUESTION 2 – DETAILED SOLUTION

Solution: Minimax Algorithm with Alpha-Beta Pruning

The Minimax Algorithm is used in two-player, zero-sum games to choose the move that gives the best guaranteed outcome, assuming the opponent also always plays optimally. The MAX player tries to maximise the final utility value while the MIN player tries to minimise it.

Alpha-Beta Pruning is a refinement that skips evaluating branches which cannot possibly change the final decision, using two running bounds: Alpha (α), the best value MAX can already guarantee, and Beta (β), the best value MIN can already guarantee along the branch currently being explored.

Initially, Alpha (α) = $-\infty$ and Beta (β) = $+\infty$.

The game tree for this problem has one MAX node, two MIN nodes, and six terminal (leaf) nodes.

Step 1 – Evaluate Left MIN Node

Leaf values: 3, 5, 6

The MIN player always keeps the smallest value seen so far:

$$\min(3) = 3$$

$$\min(3, 5) = 3$$

$$\min(3, 5, 6) = 3$$

Selected Value = 3

This value of 3 is returned to the MAX node, which updates $\alpha = \max(-\infty, 3) = 3$.

Step 2 – Evaluate Right MIN Node

Leaf values: 9, 1, 2

β starts at $+\infty$ for this branch.

Evaluate 9: running minimum = 9, so $\beta = \min(\infty, 9) = 9$. Since $\beta(9) > \alpha(3)$, evaluation continues to the next leaf.

Evaluate 1: running minimum = $\min(9, 1) = 1$, so $\beta = \min(9, 1) = 1$. Now $\beta = 1$ and $\alpha = 3$, and since $\beta \leq \alpha$ ($1 \leq 3$), the pruning condition is satisfied.

Because the pruning condition has been met, the remaining leaf (value 2) is not evaluated – no value it could hold would change what MIN ultimately returns, since MIN has already found a value (1) smaller than what MAX would accept from this branch (3).

Pruning

The leaf node with utility value 2 is pruned. This avoids one unnecessary evaluation while still producing the correct Minimax result.

Values Returned

Left MIN = 3

Right MIN = 1

MAX Node Evaluation

MAX chooses $\max(3, 1) = 3$.

Final Minimax Value

3

Best Move

The MAX player selects the left subtree, since it returns the higher utility value (3 versus 1).

Alpha-Beta Updates

Step	Alpha	Beta	Action
Start	$-\infty$	$+\infty$	Initialize
Left MIN	$-\infty$	3	Return 3
MAX	3	$+\infty$	Update α
Right MIN (9)	3	9	Continue
Right MIN (1)	3	1	$\beta \leq \alpha$
Remaining Node	-	-	Pruned / Stop Evaluation

Analysis

Without Alpha-Beta Pruning, all 6 leaf nodes would need to be evaluated to reach a decision. With Alpha-Beta Pruning, only 5 leaf nodes are evaluated, since the sixth (value 2) is skipped once β falls to or below α . This saving becomes far more significant in larger game trees, where entire unexplored subtrees – not just single leaves – can be pruned in one step.

Final Result

Best Move : Left Subtree

Final Minimax Value : 3

Pruned Node : 2

The Minimax Algorithm with Alpha-Beta Pruning correctly determines the optimal move for the MAX player while reducing the number of node evaluations needed, without changing the final outcome that plain Minimax would have produced.